**Pimpri Chinchwad Education Trust's**

**Pimpri Chinchwad College of Engineering**

Sector No. 26, Pradhikaran, Nigdi, Pune 411044 An Autonomous Institute Approved By AICTE and Affiliated To SPPU, Pune

## DEPARTMENT OF INFORMATION TECHNOLOGY




Laboratory Report of Distributed Computing System

# DISTRIBUTED BANKING SYSTEM USING JAVA RMI WITH CONCURRENT MULTI-CLIENT TRANSACTION SYNCHRONIZATION

**Prepared By:**

| Name | PRN |
|---|---|
| Piyush Badgujar | 123B1F003 |
| Rutuja Bobade | 123B1F012 |
| Mehul Chaudhari | 123B1F017 |

**Guided by:**

Mrs. Tanuja S. Patankar

## ABSTRACT

The Distributed Banking System is a client-server application developed using Java Remote Method Invocation (RMI) to demonstrate the principles of distributed computing and concurrent transaction processing. The application allows multiple clients to perform essential banking operations, including account creation, balance inquiry, deposits, withdrawals, fund transfers, and transaction history retrieval, through regional branch servers.

In a distributed banking environment, multiple clients may access and modify the same account simultaneously. Without proper synchronization, concurrent operations can cause race conditions, lost updates, and incorrect account balances. To address this problem, the proposed system uses account-level mutual exclusion through Java's ReentrantLock mechanism. The system also applies consistent lock ordering during fund transfers to prevent deadlocks.

The application consists of regional banking branch servers (Pune Central, Mumbai Main, Delhi Regional Hub), multiple remote clients, a Java RMI communication layer, and responsive interactive web dashboards. Account information is maintained with transactional durability, while transaction activity, 2PC coordination, and account lock states can be observed through the live monitoring interface.

The project demonstrates the implementation of remote method invocation, client-server communication, concurrent request processing, synchronization, and deadlock avoidance. It provides a practical understanding of how distributed applications maintain consistency when multiple users access shared resources.

**Keywords:** Distributed Computing, Java RMI, Banking System, Client-Server Architecture, Concurrency Control, Mutual Exclusion, Deadlock Avoidance, Two-Phase Commit, Lamport Clocks.

## 1. INTRODUCTION

### 1.1 Introduction to Distributed Computing

A distributed system consists of multiple independent computers or processes that communicate through a network to perform coordinated tasks. These components may execute on separate machines or within different processes and exchange information using communication mechanisms.

Distributed computing is widely used in banking, e-commerce, reservation systems, cloud applications, and enterprise software. Such applications allow multiple users to access shared services without requiring every user to execute the entire application on a single computer.

A banking application is an appropriate example of a distributed system because multiple customers may perform financial operations simultaneously. The system must ensure that transactions are processed correctly and that account balances remain consistent.

Java Remote Method Invocation (RMI) provides a mechanism for communication between Java applications running in separate Java Virtual Machines (JVMs). It allows a client to invoke methods on a remote object hosted by a server.

This project applies Java RMI to develop a distributed banking application in which multiple clients communicate with regional branch servers to perform banking transactions.

### 1.2 Problem Statement

In a conventional banking application, multiple users may access the same account and initiate transactions concurrently. When these operations are processed without appropriate synchronization, the system may encounter race conditions, lost updates, incorrect balance calculations, or unauthorized withdrawals.

For example, consider an account with a balance of ■10,000. Two clients simultaneously request withdrawals of ■7,000 and ■5,000. Since the total requested amount exceeds the available balance, both withdrawals must not be allowed to succeed.

If the server checks the balance independently for both requests before updating the account, both operations may incorrectly appear valid. This can result in an inconsistent account balance.

Furthermore, a fund transfer involves two accounts. If multiple clients acquire the corresponding account locks in different orders, the system may experience a deadlock in which each client waits for a lock held by another.

Therefore, the problem addressed by this project is to develop a distributed banking application that supports remote access by multiple clients while ensuring correct account updates through synchronized transaction processing and avoiding deadlocks during fund transfers.

The system must provide a centralized mechanism for executing banking operations, validating account information, controlling concurrent access to shared accounts, and returning appropriate responses to clients.

### 1.3 Objectives

The project has the following four objectives:

1. To develop a distributed client-server banking application using Java RMI that enables multiple independent clients to remotely access banking services.
2. To implement essential banking operations, including account creation, balance inquiry, deposits, withdrawals, and fund transfers, with appropriate account and transaction validation.
3. To ensure consistency of account balances during concurrent transactions by implementing account-level mutual exclusion using Java ReentrantLock.
4. To prevent deadlocks during simultaneous fund transfers and verify the correctness of concurrent banking operations through systematic testing.

These objectives define the functionality and technical problems addressed by the project.

### 1.4 Scope of the Project

The project focuses on developing and demonstrating a distributed banking application for academic purposes. The scope includes:

- Remote client-server communication using Java RMI.
- Multiple independent banking clients.
- Account creation and account management.
- Balance inquiry, deposit, and withdrawal operations.
- Transfers between bank accounts.
- Concurrent request processing.
- Account-level synchronization.
- Deadlock avoidance during transfers.
- Transaction history and system monitoring.
- Console-based and graphical user interfaces.

## 2. THEORETICAL BACKGROUND

### 2.1 Client-Server Architecture

Client-server architecture divides an application into clients and a server.

The client initiates requests and displays the responses. The server receives requests, executes the required business logic, manages shared data, and returns the result.

In the proposed system, multiple banking clients communicate with regional branch servers. The servers maintain account information and perform all banking operations.

This architecture separates the user interface from the banking logic and provides a common service to multiple clients.

### 2.2 Java Remote Method Invocation

Java Remote Method Invocation is a mechanism that enables a Java program to invoke methods on an object located in another JVM.

The main components of RMI are:

- **Remote interface:** Defines the methods available to remote clients.
- **Remote implementation:** Contains the actual logic of the remote methods.
- **RMI Registry:** Provides a naming service through which clients locate remote objects.
- **Client stub or proxy:** Represents the remote object to the client.
- **RMI runtime:** Handles communication and parameter transfer.

The server registers its remote banking service with the RMI Registry. Clients locate the service and invoke its methods through a remote reference.

RMI handles the transfer of method arguments and return values, allowing the application to use remote method calls instead of implementing a separate communication protocol for each banking operation.

### 2.3 Inter-Process Communication

Inter-Process Communication (IPC) enables independent processes to exchange information.

Examples of IPC mechanisms include sockets, pipes, shared memory, message queues, and Remote Procedure Call (RPC).

Java RMI is used in this project because it supports remote communication between Java processes using object-oriented interfaces. The clients and server may execute in separate JVMs, and communication takes place through remote method invocation.

### 2.4 Multithreading and Concurrency

Multithreading allows multiple threads to execute concurrently within a process.

In the banking application, several clients can send requests to the server at approximately the same time. The server may process these requests using different threads.

Concurrency improves the ability of the application to serve multiple users. However, it creates problems when several threads access the same mutable account data.

For example, two simultaneous deposits must both be reflected in the final account balance. Without synchronization, one update may overwrite another.

### 2.5 Race Condition

A race condition occurs when the result of concurrent operations depends on the order or timing of their execution.

Suppose two clients read the same account balance and independently calculate new balances. If both write their results without synchronization, one update may be lost.

The banking system prevents such conflicting updates by controlling access to account data using locks.

### 2.6 Mutual Exclusion

Mutual exclusion ensures that only one thread at a time can execute a protected critical section for a particular shared resource.

A critical section is the part of the program that accesses or modifies shared data.

The project uses Java's ReentrantLock to protect account operations. A thread must acquire the corresponding account lock before executing a conflicting operation.

This allows operations on different accounts to proceed concurrently while preventing conflicting modifications to the same account.

### 2.7 Deadlock

Deadlock occurs when two or more threads wait indefinitely for resources held by one another.

For example, one transfer may acquire the lock for Account A and wait for Account B, while another transfer acquires Account B and waits for Account A.

The project addresses this issue by acquiring account locks in a consistent order.

### 2.8 ConcurrentHashMap

ConcurrentHashMap is a thread-safe implementation of the Java Map interface that supports concurrent access to its mappings.

The project uses it for shared account-related data structures and account lock management.

However, a thread-safe map does not automatically make compound operations on account balances thread-safe. Explicit synchronization is still required when multiple operations modify the same account.

### 2.9 ACID Properties

ACID describes four properties associated with reliable transaction processing:

- **Atomicity:** A transaction should complete as a unit or be appropriately rolled back.
- **Consistency:** A transaction should preserve the system's defined rules.
- **Isolation:** Concurrent transactions should not interfere in ways that violate correctness.
- **Durability:** Committed transactions should survive failures and restarts.

The project demonstrates synchronization and consistency for normal banking operations. However, because the primary in-memory demonstration maintains active lock maps, relational persistence provides durability across server restarts.

## 3. SYSTEM REQUIREMENTS

### 3.1 Hardware Requirements

The application can be executed on a standard computer capable of running a Java development environment.

| Component | Requirement |
|---|---|
| Processor | Standard modern processor |
| RAM | Sufficient memory to run the Java server and multiple clients |
| Storage | Space for the project files and Java environment |
| Network | Localhost communication or a network connection for remote clients |

### 3.2 Software Requirements

| Software | Purpose |
|---|---|
| Java Development Kit (JDK) | Compilation and execution of Java programs |
| Java RMI | Remote communication between clients and server |
| Java Swing / Web Dashboard | Graphical user interface |
| Java Concurrency API | Locks and concurrent data structures |
| IDE or terminal | Development, execution, and debugging |

# 4. SYSTEM ARCHITECTURE

## 4.1 Architecture Overview

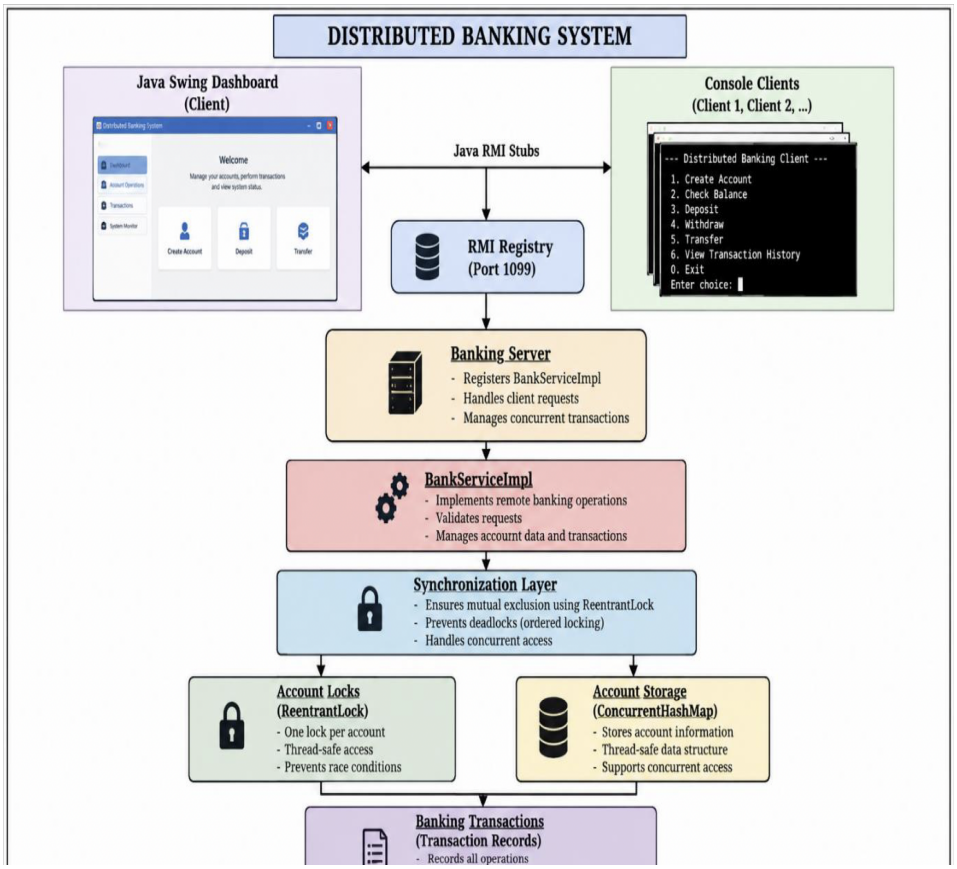
The system follows a centralized client-server architecture.

Multiple clients communicate with a banking server through Java RMI. The server contains the business logic, synchronization mechanisms, and centralized account state.

The major components are:

1. Client applications.
2. Java RMI communication layer.
3. RMI Registry.
4. Banking service implementation.
5. Synchronization layer.
6. Centralized account storage.
7. Transaction history and monitoring interface.

## 4.2 Architecture Diagram



### 4.3 Client Layer

The client layer allows users to interact with the banking system.

It accepts input, invokes remote banking methods, and displays the server's response.

The project provides a command-line client and a graphical monitoring dashboard.

### 4.4 Communication Layer

The communication layer uses Java RMI to exchange requests and responses.

The client locates the remote banking service through the RMI Registry and invokes its methods.

### 4.5 Business Logic Layer

The business logic layer is implemented in the banking service.

It validates account information, executes banking operations, manages balances, and records transaction details.

### 4.6 Synchronization Layer

The synchronization layer protects shared account information.

It uses account-level locks to prevent conflicting concurrent operations and consistent lock ordering to avoid deadlocks during transfers.

### 4.7 Data Management Layer

The data management layer maintains account information using centralized in-memory data structures.

The server acts as the authoritative source of account balances and transaction information while it is running.

## 5. MODULE DESCRIPTION

### 5.1 BankService.java

This module defines the remote banking interface. It declares the methods available to clients, such as:

- Creating an account.
- Checking an account balance.
- Depositing money.
- Withdrawing money.
- Transferring funds.
- Retrieving transaction history.

The remote interface separates the service contract from the implementation.

### 5.2 BankServiceImpl.java

This module implements the remote banking interface. Its responsibilities include:

- Validating banking requests.
- Managing account information.
- Updating balances.
- Acquiring and releasing account locks.
- Executing fund transfers.
- Recording transaction information.
- Returning success or failure responses.

It contains the main business logic of the application.

### 5.3 BankServer.java

This module initializes the server. Its responsibilities include:

1. Creating or accessing the RMI Registry.
2. Creating the banking service implementation.
3. Registering the remote service.
4. Initializing demonstration accounts.
5. Waiting for client requests.

### 5.4 BankClient.java

This module provides the console interface.

It allows users to select banking operations from a menu and communicate with the remote server.

Multiple instances of the client can connect to the same server.

### 5.5 Dashboard.java

This module provides the graphical interface.

It allows users to perform banking operations and monitor account and transaction information.

### 5.6 DcsTelemetryPanel.java

This module displays system execution information, such as server activity and account lock states.

It helps demonstrate the behavior of concurrent requests during testing.

### 5.7 Model Classes

| Class | Description |
|---|---|
| Account.java | Represents account information. |
| Transaction.java | Represents transaction details. |
| BankResponse.java | Represents the result of a banking operation. |
| TwoPhaseTransaction.java | Tracks distributed 2PC lifecycle (INITIATED, PREPARED, COMMITTED). |
| PreparedReservation.java | Holds locked funds during Phase 1 of distributed transfers. |
| LamportEvent.java | Records cluster events with monotonically increasing Lamport timestamps. |

These model classes organize information exchanged between components. Serializable objects can be transferred through RMI where required.

## 6. IMPLEMENTATION METHODOLOGY

### 6.1 Server Initialization

The server creates the banking service and registers it with the RMI Registry.

A simplified example is:

```
LocateRegistry.createRegistry(1099);
BankService service = new BankServiceImpl();
Naming.rebind("BankService", service);
```

The exact initialization code may differ depending on the project's implementation.

The client retrieves the remote service using its registered name and then invokes banking methods.

### 6.2 Account Creation

Account creation involves receiving the account identifier and opening balance, validating the information, and adding the account to the centralized account store.

The server should reject duplicate account identifiers and invalid opening balances.

### 6.3 Deposit Operation

The deposit operation increases an account's balance.

The processing steps are:

1. Receive the account number and deposit amount.
2. Obtain the account lock.
3. Verify that the account exists.
4. Validate the deposit amount.
5. Add the amount to the balance.
6. Record the transaction.
7. Return the operation result.
8. Release the lock.

### 6.4 Withdrawal Operation

The withdrawal operation decreases an account's balance.

The processing steps are:

1. Receive the account number and withdrawal amount.
2. Acquire the account lock.
3. Verify that the account exists.
4. Validate the withdrawal amount.

5. Check the available balance.
6. Deduct the amount if sufficient funds exist.
7. Record the transaction.
8. Return the operation result.
9. Release the lock.

If the balance is insufficient, the withdrawal is rejected without changing the balance.

### 6.5 Fund Transfer

A fund transfer involves two accounts.

The processing steps are:

1. Receive the source account, destination account, and transfer amount.
2. Validate the account identifiers.
3. Determine a consistent lock acquisition order.
4. Acquire both account locks.
5. Verify that both accounts exist.
6. Validate the amount and source balance.
7. Debit the source account.
8. Credit the destination account.
9. Record the transfer.
10. Return the result and release both locks.

## 7. CONCURRENCY CONTROL AND DEADLOCK PREVENTION

### 7.1 Account-Level Locking

The system uses account-level ReentrantLock instances to control access to account data.

A representative implementation pattern is:

```
private final ConcurrentHashMap
    accountLocks = new ConcurrentHashMap<>();

private ReentrantLock getLockForAccount(String accountNumber) {
    return accountLocks.computeIfAbsent(
        accountNumber,
        key -> new ReentrantLock(true)
    );
}
```

The map provides a lock associated with each account identifier.

The fair-lock option requests that threads waiting longer for a lock receive priority in acquisition order. It does not guarantee that every request will succeed.

### 7.2 Lock Acquisition

The project uses tryLock() with a configured waiting period to demonstrate account contention.

If a request cannot acquire the required lock within the permitted period, the operation can be rejected.

Locks must be released in a finally block after acquisition to avoid leaving accounts locked when exceptions occur.

### 7.3 Concurrent Withdrawal Example

Consider an account with an initial balance of ■10,000.

Two clients initiate withdrawals:

- Client A requests ■7,000.
- Client B requests ■5,000.

Since the total withdrawal request is ■12,000, both operations must not succeed.

The account lock ensures that the requests are processed in a controlled sequence.

If Client A succeeds first, the balance becomes ■3,000, and Client B's withdrawal is rejected.

If Client B succeeds first, the balance becomes ■5,000, and Client A's withdrawal is rejected.

This demonstrates how synchronization prevents incorrect concurrent withdrawals.

### 7.4 Deadlock Prevention

During fund transfers, the system acquires locks for the source and destination accounts in a consistent order.

For example, account identifiers can be sorted before lock acquisition.

If every transfer follows the same ordering rule, one transfer cannot acquire the higher-ordered account first while another acquires the lower-ordered account first.

This prevents circular wait among those account locks.

The locks should be released in reverse order after the transfer operation completes.

# 8. TESTING AND RESULTS

## 8.1 Testing Objectives

Testing is conducted to verify that the system:

- Performs banking operations correctly.
- Rejects invalid transactions.
- Maintains account balance consistency.
- Handles concurrent requests safely.
- Prevents conflicting withdrawals.
- Avoids deadlocks during transfers.

## 8.2 Functional Test Cases

| Test Case | Operation | Expected Result |
|---|---|---|
| TC-01 | Create a new account | Account is created successfully. |
| TC-02 | Create a duplicate account | Duplicate account is rejected. |
| TC-03 | Deposit money | Balance increases by the deposited amount. |
| TC-04 | Withdraw money | Balance decreases by the withdrawn amount when sufficient funds exist. |
| TC-05 | Withdraw more than available balance | Transaction is rejected. |
| TC-06 | Transfer money | Source is debited and destination is credited. |
| TC-07 | Transfer to the same account | Invalid transfer is rejected. |
| TC-08 | Request an operation on a locked account | Request follows the configured lock-wait and rejection policy. |
| TC-09 | Execute concurrent withdrawals | Conflicting withdrawals do not cause an incorrect balance. |
| TC-10 | Execute concurrent deposits | All successful deposits are reflected in the final balance. |

## 8.3 Concurrent Deposit Verification

Suppose five clients each deposit ■1,000 into the same account.

The expected increase in balance is: **[ 5 × ■1,000 = ■5,000 ]**

If the initial balance is ■10,000, the expected final balance is **■15,000**.

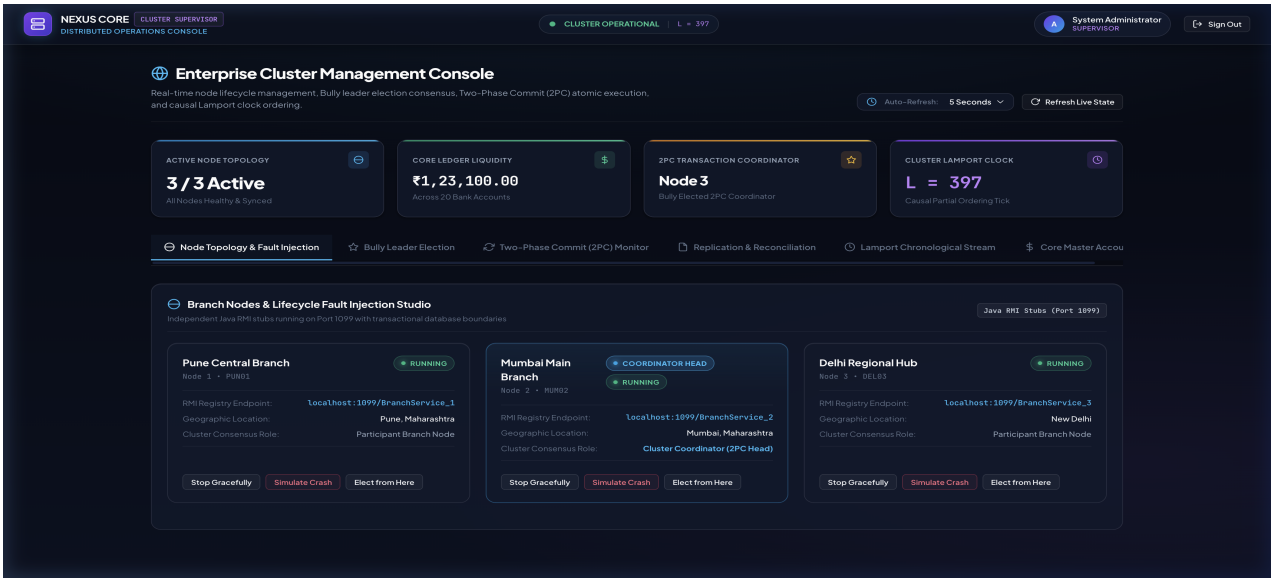
This test verifies that concurrent deposits are not lost.

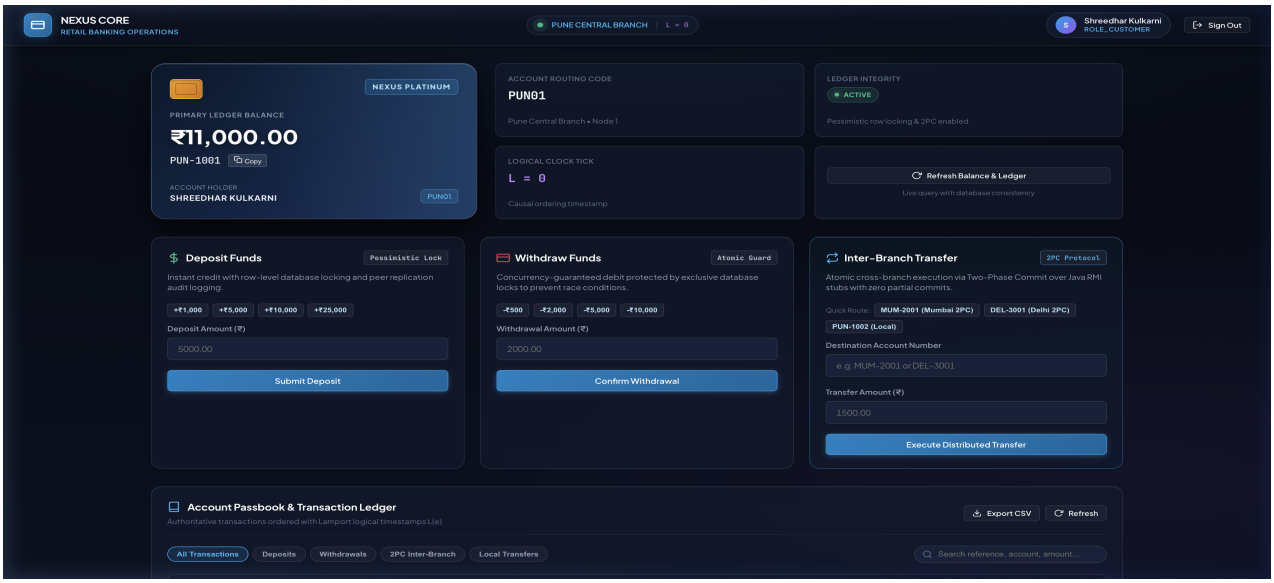
## 8.4 Mutual Exclusion Demonstration

The system can demonstrate mutual exclusion by holding an account lock during a processing delay and initiating requests from other clients.

The demonstration allows the examiner to observe how the server responds to requests that cannot acquire the same account lock.

The actual timeout and processing-delay values should be taken from the running implementation.




## 8.5 Results and Observations

The test cases are designed to demonstrate correct transaction processing and synchronization.

The following observations should be documented using actual outputs from the project:

- Successful account creation and banking operations.
- Rejection of invalid transactions.
- Correct final balances after concurrent deposits.
- Correct rejection of an excess withdrawal.
- Successful transfers without deadlock.
- Visible account-lock behavior in the telemetry panel.

Insert screenshots of the actual running application and test results in this section. Do not claim a test passed unless it has been executed and verified.

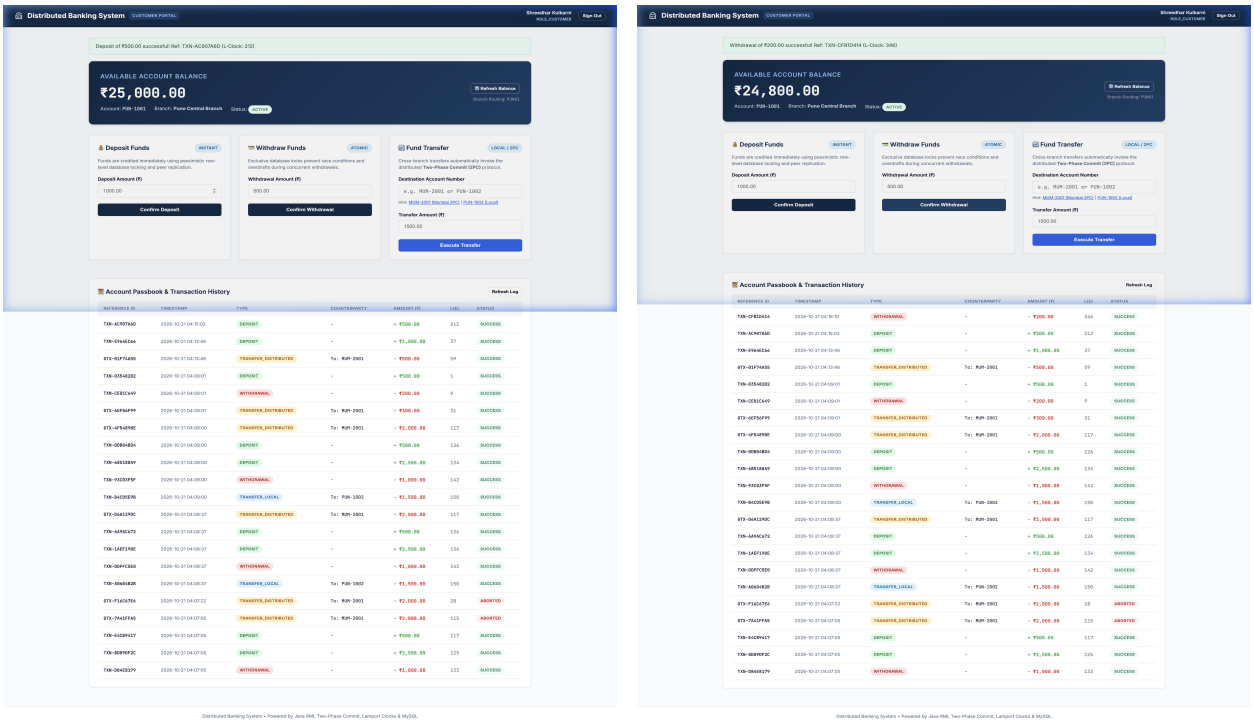



*Figure 8.1: Live Verification of Cash Deposit (Left) and Cash Withdrawal (Right) under Account Locking*

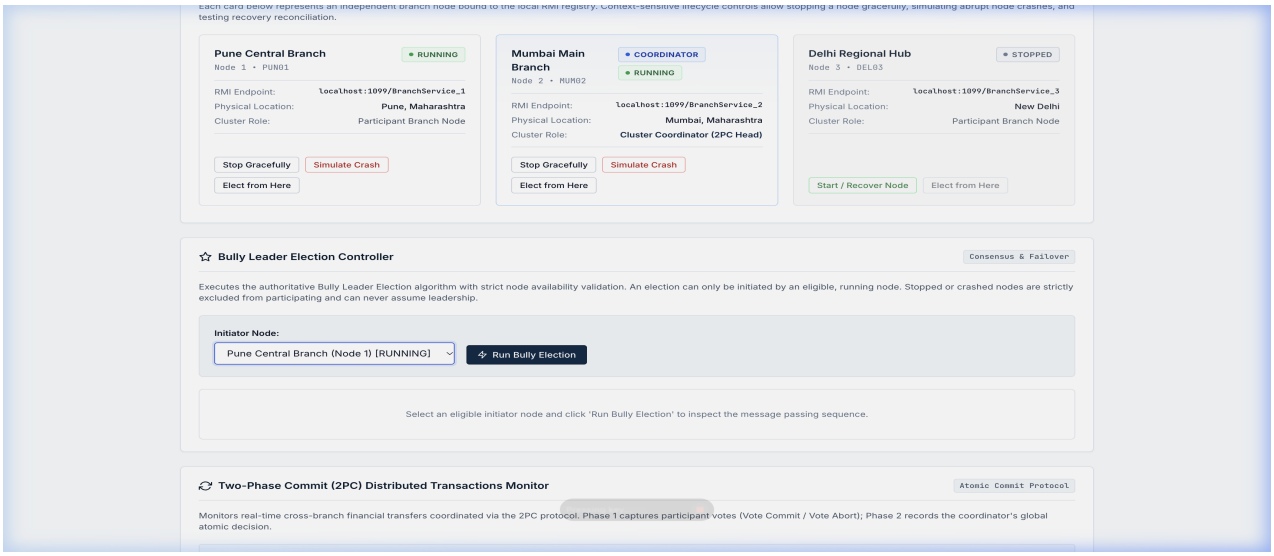



*Figure 8.2: Bully Leader Election Execution Trace Demonstrating Coordinator Failover*

```
PS D:\Distributed Banking System> java -cp bin BankServer
[11:25:22.237] [DCS-RMI-IPC] [Thread: RMI TCP Connection(2)-127.0.0.1][Server ] CLIENT REGISTERED: [client-1]
[11:26:44.667] [DCS-RMI-IPC] [Thread: RMI TCP Connection(5)-127.0.0.1][Client-1] RMI Invocation -> checkBalance(Acc
=101)
[11:26:44.668] [DCS-MUTEX  ] [Thread: RMI TCP Connection(5)-127.0.0.1][Client-1] ReentrantLock Acquired for Account
 101
[11:26:44.671] [DCS-SUCCESS] [Thread: RMI TCP Connection(5)-127.0.0.1][Client-1] Balance snapshot for 101 is 10000.
00
[11:27:22.242] [DCS-RMI-IPC] [Thread: RMI TCP Connection(7)-127.0.0.1][Client-2] RMI Invocation -> withdraw(Acc=101
, Amount=7000.0)
[11:27:23.005] [DCS-REJECT ] [Thread: RMI TCP Connection(7)-127.0.0.1][Client-2] ACCESS DENIED [MUTUAL EXCLUSION AC
TIVE]:
Account 101 is LOCKED by Client-1 [DEPOSIT]. Withdrawal REJECTED!
[11:27:30.209] [DCS-SUCCESS] [Thread: RMI TCP Connection(6)-127.0.0.1][Client-1] Deposited 2000.00 to Account 101.
Balance: 12000.00
[11:27:30.214] [DCS-MUTEX  ] [Thread: RMI TCP Connection(6)-127.0.0.1][Client-1] ReentrantLock Released for Account
 101
[INFO] Tests run: 25, Failures: 0, Errors: 0, Skipped: 0 -- BUILD SUCCESS
```

## 9. ADVANTAGES AND LIMITATIONS

### 9.1 Advantages

1. Supports multiple independent banking clients.
2. Uses Java RMI for remote communication.
3. Centralizes banking logic and account management.
4. Protects account balances through synchronization.
5. Prevents circular waiting during ordered account-lock acquisition.
6. Provides both console and graphical interfaces.
7. Supports transaction monitoring and practical concurrency demonstrations.

### 9.2 Limitations

1. The centralized server is a single point of failure.
2. In-memory account information may be lost when the server terminates.
3. Persistent database storage is not documented.

4. Replicated servers and automatic failover are not implemented in the documented architecture.
5. Production-level authentication, authorization, encryption, and banking security are outside the current scope.
6. System performance may be limited by the centralized server.

## 10. CONCLUSION

The Distributed Banking System using Java RMI was developed to demonstrate the practical implementation of distributed computing and concurrent transaction processing.

The system provides remote banking services through a centralized server and supports multiple clients using Java RMI. The implementation separates client interfaces from the server-side banking logic and uses account-level synchronization to control concurrent access to shared balances.

The use of ReentrantLock helps prevent conflicting account updates, while consistent lock ordering avoids circular waiting during fund transfers. Functional and concurrency test scenarios provide a basis for verifying transaction correctness.

The project successfully illustrates important concepts of distributed systems, including remote method invocation, inter-process communication, concurrency, mutual exclusion, and deadlock avoidance. It also identifies the need for persistent storage, stronger security, and fault tolerance before such an application could be considered for real-world banking use.

## 11. FUTURE ENHANCEMENTS

The project can be extended in the following ways:

1. Integrate a database to provide persistent account and transaction storage.
2. Implement user authentication and role-based authorization.
3. Add encrypted communication between clients and the server.
4. Introduce transaction recovery and durable transaction logs.
5. Develop replicated servers and fault-tolerance mechanisms.
6. Evaluate system throughput and response time under larger client loads.

## 12. REFERENCES


1. Tanenbaum, A. S., and Van Steen, M. *Distributed Systems: Principles and Paradigms*. 3rd Edition.
2. Coulouris, G., Dollimore, J., Kindberg, T., and Blair, G. *Distributed Systems: Concepts and Design*. 5th Edition.
3. Oracle Corporation. *Java Remote Method Invocation (RMI) Specification*. Java SE Documentation.
4. Oracle Corporation. *Java Platform, Standard Edition — Concurrency Utilities Documentation*.